

Considerações Finais: Projeto Pet Flow

1. Avaliação de Frameworks e Tecnologias

O projeto **Pet Flow** foi concebido sob uma arquitetura distribuída moderna, visando a alta disponibilidade e a escalabilidade. A escolha da stack técnica foi pautada pela necessidade de atender múltiplos clientes (Web e Mobile) de forma eficiente:

- **Arquitetura e Backend:** A utilização de **Node.js/Express** permitiu a construção de uma API REST robusta, capaz de servir tanto a interface Web em **React** quanto a aplicação Mobile em **React Native** através de comunicação via HTTP.
- **Banco de Dados:** Optou-se pelo **PostgreSQL**, hospedado no **Supabase**, utilizando uma estrutura de tabelas isoladas por domínio (Clinic, Customer, Service). A segurança dos dados foi reforçada pela criptografia **AES-256-CBC** em informações sensíveis.
- **Desenvolvimento e Deploy:** A adoção de **TypeScript** em todas as camadas garantiu maior tipagem e segurança ao código. O fluxo de deploy foi automatizado entre **Vercel** (Frontend) e **Render** (Backend), com o versionamento centralizado no GitHub via **CI/CD**.

2. Análise Crítica e Proposta de Melhorias

Análise Crítica

O desenvolvimento em modelo **monorepo** trouxe agilidade ao versionamento e à sincronização entre o backend e os frontends. No entanto, o principal desafio técnico enfrentado pela equipe foi o gerenciamento da concorrência operacional em um sistema distribuído e a necessidade de garantir a integridade dos dados em tempo real. A implementação do JWT (JSON Web Token) foi essencial para consolidar a segurança no acesso aos dados da clínica.

Proposta de Melhorias

Para futuras iterações e escalabilidade, propomos:

- **Performance:** Implementação de uma camada de cache (ex: Redis) para otimizar consultas frequentes e reduzir a carga no banco de dados principal.
- **Arquitetura:** Caso o volume de acessos cresça, a migração para uma estrutura de microsserviços permitiria maior independência e escalabilidade individual de módulos (ex: separação do módulo financeiro).
- **Monitoramento:** Aprofundar a instrumentação da aplicação com ferramentas de monitoramento de performance (APM) para identificar gargalos de latência de forma proativa.

3. Gestão do Trabalho e Contribuições

O desenvolvimento foi acompanhado por um controle centralizado no GitHub, garantindo a colaboração síncrona dos membros da equipe.

- **Quadro Visual de Desenvolvimento:** O progresso do projeto foi estruturado para

garantir a entrega das funcionalidades conforme os requisitos estabelecidos.

- Nota: O gráfico de "Commits over time" demonstra o ritmo de desenvolvimento constante, com picos de produtividade entre abril e maio de 2026, refletindo o período de maior intensidade na codificação das funcionalidades principais.
- **Equipe e Atribuições:**
 - **Equipe:** Anna Carolina Saldanha de Lima, Cristiano Henrique Amorim, Júlia Persson Mascari, Lucas Soares dos Reis, Natália Kiefer Ferreira, Rafael Vinícius da Silva, Vinicius Henrique de Oliveira Neves.
 - *Responsabilidades:* A equipe atuou de forma interdisciplinar, com foco na integração da API REST com os clientes Web e Mobile, mantendo a consistência da stack TypeScript em todo o ciclo de vida do software.